

halfedge：顶点焊接模块设计文档

src/weld.rs

halfedge 库

2026-07-04

目录

1	模块定位	1
1.1	API	2
2	算法	2
2.1	整体流程	2
2.2	排序 + 双重循环: $O(n \log n + nk)$	2
2.3	Union-Find 等价类	2
2.4	重映射半边 vertex 引用	3
2.5	清理退化面	3
2.6	最终校验	3
3	复杂度	4
4	退化守卫	4
5	测试覆盖	4
6	设计权衡	4

1 模块定位

weld.rs 提供 `weld_vertices`: 按欧氏距离阈值合并「位置相同」的顶点, 常用于修复 OBJ/PLY 载入时因浮点精度产生的「裂缝」——同一物理顶点被表示为多个微小偏差的副本。

1.1 API

签名 `pub fn weld_vertices(mesh: &mut MeshStorage, epsilon: f64)`
`-> Result<usize, TopologyError>`

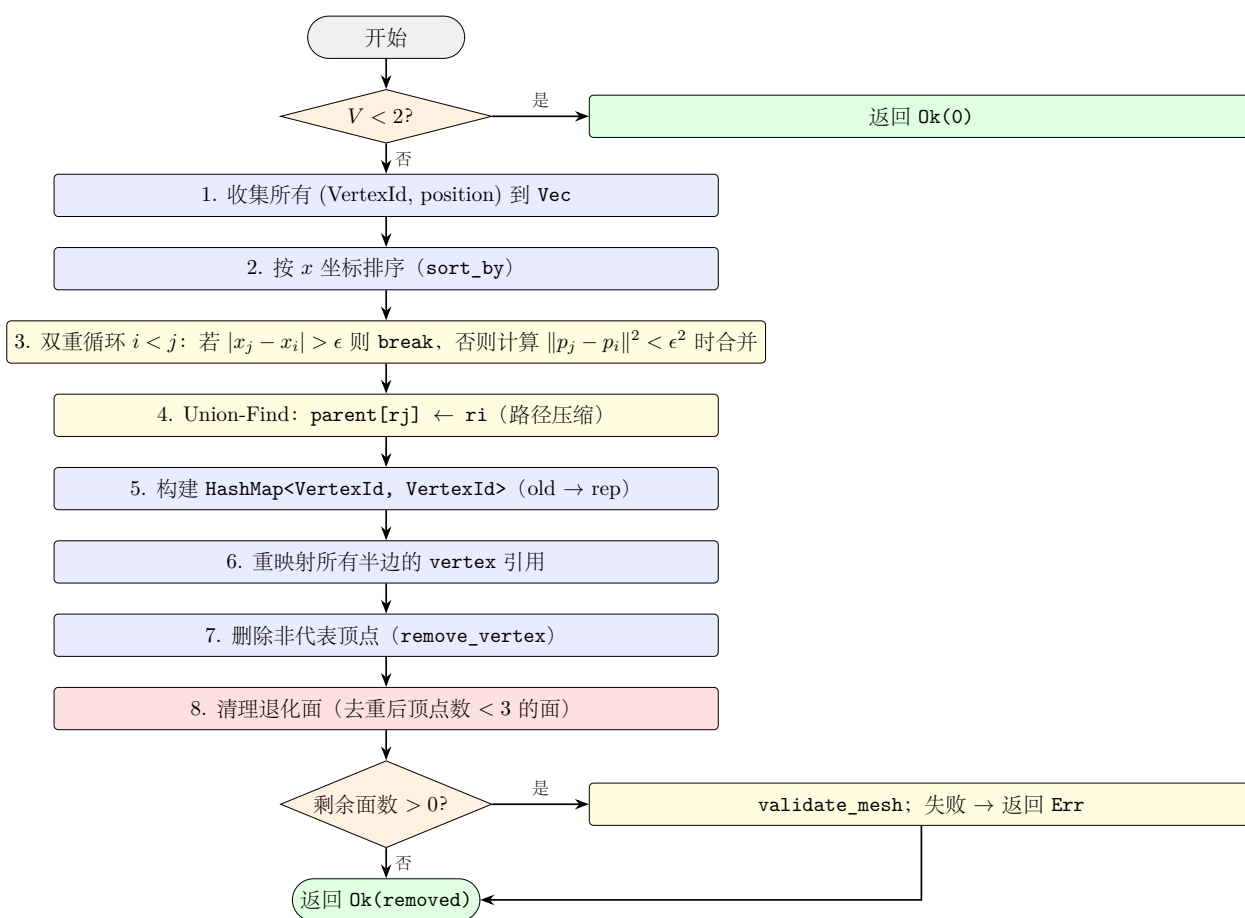
输入 `mesh`: 可变借用; `epsilon`: 合并阈值 (≥ 0)

输出 成功时返回被移除的顶点数量

错误 修复后拓扑校验失败时返回 `TopologyError`

2 算法

2.1 整体流程



2.2 排序 + 双重循环: $O(n \log n + nk)$

直接两两比较所有顶点对是 $O(n^2)$, 对大网格不可接受。本实现按 x 坐标排序后, 内层循环遇到 $|x_j - x_i| > \epsilon$ 即 `break`, 将平均复杂度降至 $O(n \log n + nk)$, 其中 k 是 x 邻域内的平均顶点数 (对均匀分布网格 $k \approx$ 常数)。

2.3 Union-Find 等价类

合并操作使用并查集 (Union-Find):

```

1 let mut parent: Vec<usize> = (0..n).collect();
2 // find with path halving
3 let mut root = i;
4 while parent[root] != root { root = parent[root]; }
5 // union: parent[rj] = ri (ri i , rj j )
6 if ri != rj { parent[rj] = ri; }

```

代表元：等价类中 x 最小的顶点（因为外层循环 i 升序，首次合并时 $i < j$, $\text{parent}[rj] = ri$ 保证代表元是序号最小的）。

2.4 重映射半边 vertex 引用

收集 `mapping: HashMap<VertexId, VertexId> (old  $\rightarrow$  rep)` 后，遍历所有半边：

```

1 for he in mesh.halfedge_ids().collect::

```

2.5 清理退化面

合并后，可能产生顶点重复的退化面（如原 $[a, b, c]$ 中 a, b 被合并为 a ，则面变为 $[a, a, c]$ ，唯一顶点数 $= 2 < 3$ ）。对这些面：

1. 收集面的所有半边；
2. 逐条 `remove_halfedge`（同时解除 `twin` 关系）；
3. `remove_face`。

2.6 最终校验

调用 `topology_ops::validate_mesh` 检查修复后网格的拓扑合法性（`twin` 互指、`next/prev` 一致、面边界环闭合等）。失败时返回 `Err`。

3 复杂度

阶段	时间复杂度	备注
排序	$O(n \log n)$	$n = V$
双重循环	$O(nk)$	$k = x$ 邻域平均顶点数
Union-Find	$O(n\alpha(n))$	$\alpha =$ 反 Ackermann
重映射半边	$O(E)$	遍历所有半边
删除冗余顶点	$O(V)$	
清理退化面	$O(F \cdot \bar{k})$	$\bar{k} =$ 平均面边数
validate_mesh	$O(V + E + F)$	线性校验
总计	$O(n \log n + nk + E)$	通常 k 很小, 接近 $O(n \log n)$

4 退化守卫

- $V < 2$ 时直接返回 `Ok(0)`, 跳过所有处理;
- `partial_cmp` 返回 `None (NaN)` 时使用 `Equal` 避免 panic;
- `remove_vertex` 内部解除该顶点的所有半边引用, 安全;
- `contains_halfedge` / `contains_face` 防止重复删除;
- 修复后 `validate_mesh` 兜底, 确保拓扑合法。

5 测试覆盖

测试名	验证点
weld_no_vertices	空网格返回 <code>Ok(0)</code>
weld_single_vertex	单顶点返回 <code>Ok(0)</code>
weld_two_close_vertices	距离 0.001 的两顶点在 $\epsilon = 0.01$ 下合并为 1, 第三个顶点保留

全模块共 3 个单元测试, 覆盖空网格、单顶点、合并三个核心场景; 项目整体 `cargo test` 共 371 个用例。**待补充**: 退化面清理测试、大规模网格性能测试、 $\epsilon = 0$ 严格相等测试。

6 设计权衡

- **排序 + 双重循环 vs 空间哈希**: 当前实现简单稳健, 对均匀分布顶点接近 $O(n \log n)$; 若顶点高度聚集 (k 大), 可改用空间哈希 (uniform grid) 将 k 限制为常数;
- **Union-Find vs 直接覆盖**: 并查集支持**传递合并** ($a \sim b, b \sim c \Rightarrow a \sim c$), 避免「链式合并」错误, 而直接覆盖可能丢失等价类信息;

- **仅按 x 排序**: 对极端情况（所有顶点 x 相同但 y, z 不同）退化为 $O(n^2)$ ；可改为按最长轴排序，但实现复杂度增加；
- **代表元选择**: 选择 x 最小的顶点（序号最小），保证幂等性（多次焊接结果相同）；不选择「最近邻」中心点，因为需要计算新位置，增加复杂度；
- **不更新 `Vertex.halfedge` 引用**: `remove_vertex` 内部已处理解除引用，剩余顶点的入口字段保持有效；若代表元 vertex 的 halfedge 引用指向被删除的顶点相关半边，`validate_mesh` 会报告。